

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 — Ağ Tabanlı Paralel Dağıtım Sistemleri
Proje 7: Veritabanı Yedekleme ve Otomasyon

Selim Bedirhan Öztürk

20291277

GitHub: <https://github.com/selimbedirhan/BLM4522-Ag-Tabanlı-Paralel-Dagitim-Sistemleri>

ÖZET

Bu projede PostgreSQL 14 üzerinde Hastane Yönetim Sistemi veritabanı tasarlanmış ve bu kritik veri altyapısı için Kurumsal Seviye (Enterprise-Grade) bir "Otomatik Yedekleme ve Denetim Sistemi" geliştirilmiştir. Proje; günlük/haftalık/aylık otomatik yedekleme senaryolarını (Custom ve SQL formatlarında), veritabanı içinde çalışan mantıksal bir denetim kayıt sistemini (`yedekleme_log`), boyut anomalilerini ve başarısız yedekleri yakalayan uyarı mekanizmasını, eski yedeklerin diski doldurmasını engelleyen "Retention Policy" (Rotasyon) kurallarını, alınan yedeklerin bütünlüğünü test eden otomatik doğrulama (Verification) altyapısını ve son olarak yöneticiler için üretilen HTML tabanlı performans raporlama modüllerini kapsamaktadır.

1. Giriş ve Motivasyon

Bilişim sistemlerinde karşılaşılabilecek donanım arızaları, fidye yazılımları (Ransomware), kullanıcı hataları (yanlışlıkla `DROP TABLE` veya `DELETE` komutlarının çalıştırılması) gibi felaket senaryolarına karşı verinin korunmasının tek kesin yolu düzenli ve güvenilir bir yedekleme altyapısına sahip olmaktır.

Bu projenin temel motivasyonu, veritabanı yedekleme işlemini sadece "bir dosya oluşturup bir kenara bırakmak" vizyonundan çıkarıp, şu prensiplerle çalışan tam otomatik bir sisteme dönüştürmektir:

- **Otomasyon:** İnsan müdahalesine gerek kalmadan planlanan zamanlarda yedeğin alınması.
- **İzlenebilirlik (Audit):** Her yedekleme işleminin boyut, süre, durum gibi metriklerinin veritabanına loglanması.
- **Doğrulama (Verification):** Alınan yedeklerin gerçekten "geri dönülebilir" olduğunun sürekli test edilmesi.
- **Optimizasyon (Rotation):** Sunucu diskinin dolmasını önlemek için eski yedeklerin kural bazlı silinmesi.
- **Proaktif Uyarı:** Olası bir hata veya anomali (örn. yedeğin bir anda yarı yarıya küçülmesi) durumunda sistem yöneticilerine uyarı gönderilmesi.

2. Ortam ve Kullanılan Teknolojiler

2.1 Kullanılan Araçlar

Araç / Teknoloji	Türü	Açıklama
PostgreSQL 14	VTYS	Açık kaynak veritabanı yönetim sistemi
pg_dump	Yedekleme	PostgreSQL'in yerleşik yedekleme aracı (Custom ve Plain formatlar)
pg_restore	Geri Yükleme	Veritabanını yedekten kurtarma ve içerik okuma
Bash Scripting	Otomasyon	Tüm operasyonel süreçlerin (yedek, rotasyon, test) yönetimi
macOS osascript	Bildirim	İşletim sistemi seviyesinde GUI bildirimleri (Notification)
Cron / Launchd	Görev Yöneticisi	İşletim sisteminin zamanlanmış görev tetikleyicisi
HTML5 / CSS	Raporlama	Gelişmiş dashboard tarzı rapor görünümü oluşturma

3. Hastane Veritabanı Şeması ve Denetim Tablosu

Sistem, gerçek hayattaki karmaşık ilişkileri yansıtmayı amacıyla bir **Hastane Yönetim Sistemi** üzerinden modellendirilmiştir.

3.1 Şema Tasarımı

Veritabanı 9 adet operasyonel tablodan oluşmaktadır:

- bolumlar** : Hastane klinikleri
- doktorlar** : T.C. kimlik, branş ve özlük bilgileri
- hastalar** : Kimlik, demografik ve kan grubu verileri
- randevular** : Hasta-Doktor randevu eşleşmeleri ve durum takibi
- muayeneler** : Şikayet, tanı ve uygulanan tedaviler
- yatislar** : Yatan hasta işlemleri, oda/yatak tahsisleri
- ilaclar** : Hastane eczanesi stok ve fiyatlandırması
- receteler** : Doktorun muayene sonucunda yazdığı ilaç kayıtları
- faturalar** : Verilen hizmetlerin mali boyutları

3.2 Yedekleme Denetim (Audit) Tablosu

Microsoft SQL Server'daki yerleşik **msdb** veritabanı veya Agent job izleme mekanizmalarına benzer şekilde, PostgreSQL'de bu işlemi yapmak için özel bir **yedekleme_log** tablosu tasarlanmıştır:

```
CREATE TABLE yedekleme_log (  
    log_id SERIAL PRIMARY KEY,  
    yedek_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    yedek_tipi VARCHAR(30) NOT NULL,  
    dosya_adi VARCHAR(500),  
    dosya_boyutu BIGINT,  
    sure_saniye INTEGER,  
    durum VARCHAR(20) DEFAULT 'basarili' CHECK (durum IN ('basarili', 'b  
    hata_mesaji TEXT,  
    dogrulama_durumu BOOLEAN,  
    dogrulama_tarihi TIMESTAMP,  
    veritabani_boyutu BIGINT  
);
```

Bu tablo, Bash scriptleri tarafından her işlem sonucunda otomatik doldurularak, DBA'lerin yedekleme performansını sorgulayabilmesine olanak tanır.

4. Otomatik Yedekleme Altyapısı

Projenin merkezinde bulunan `04_otomatik_yedekleme.sh` scripti, sistemin güvenliğini sağlayan ana motordur.

4.1 Çoklu Yedekleme Stratejisi

Script, çalıştırıldığı günü sistem saatinden (`date +%u` ve `date +%d`) okuyarak yedeğin tipini (`daily` , `weekly` , `monthly`) belirler ve veriyi kendi klasörüne yazar. Ayrıca iki farklı yedek aynı anda alınır:

1. **Custom Format (`pg_dump -Fc`)**: Hem sıkıştırılmış hem de `pg_restore` ile parça parça (sadece belirli bir tablonun) geri dönülmesine olanak tanıyan optimum formattır.
2. **Plain SQL Format (`pg_dump -Fp | gzip`)**: Sadece şema okumak veya standart SQL komutlarıyla manuel müdahale edebilmek amacıyla alınan arşiv niteliğindeki "Clear-text" yedek formatıdır.

4.2 Veritabanı ve İşletim Sistemi Entegrasyonu

Yedekleme tamamlandığında, dosya boyutu hesaplanır ve operasyonun başarısı `psql` komutuyla doğrudan `yedekleme_log` tablosuna `INSERT` edilir. Hata olursa `exit code` loglanır ve macOS `osascript` aracılığıyla sistemin bildirim merkezine (Notification Center) görsel uyarı gönderilir.

4.3 Boyut Anomalisi Tespiti

Çoğu sistemde yedekleme başarılı (Exit Code 0) bitse bile asıl tehlike yedeğin içinin eksik olmasıdır (Örneğin, yedek boyutu dün 5GB iken bugün 1GB ise büyük ihtimalle tablolar silinmiştir). Script, son yedeğin boyutunu okuyarak güncel boyut ile karşılaştırır. Değişim `%50` esik değerini aşarsa veritabanına `durum='uyari'` olarak log düşer ve boyut anomali bildirimi üretir.

5. Yedek Rotasyonu ve Retention Policy

Her gün alınan yedekler zamanla sunucunun diskini dolduracaktır. Diskin dolması veritabanının "Read-Only" moda geçmesine veya tamamen durmasına sebep olur. Bu sorunu çözmek için `08_yedek_rotasyon.sh` kural tabanlı bir silme işlemi uygular.

5.1 Rotasyon Kuralları (GVS - Grandfather-Father-Son)

- **Daily (Günlük):** Son 7 gün saklanır. Daha eski günlerin yedekleri silinir. (Hızlı geri dönüşler için)
- **Weekly (Haftalık):** Son 4 hafta saklanır. Eski haftalar silinir. (Orta vadeli hataları kapsamak için)
- **Monthly (Aylık):** Son 6 ay saklanır. Eski aylar silinir. (Regülasyon ve uyumluluk için)

5.2 Limit Koruma Sistemi

Zaman bazlı rotasyona ek olarak bir "Maksimum Dosya Kotası" uygulanır. Parametrik (örn: `MAX_BACKUP_COUNT=50`) limite ulaşıldığında, tarihsel sınıra bakılmaksızın en eski dosyalar silinerek diskin güvenliği mutlak suretle garanti altına alınır.

6. Otomatik Yedek Doğrulama Sistemi

"Geri yüklenemeyen yedek, yedek değildir" prensibinden yola çıkılarak `09_yedek_dogrulama.sh` ile tam otomatik (Hands-off) bir test sistemi inşa edilmiştir:

1. **Dosya Bütünlüğü:** `pg_restore --list` komutu kullanılarak yedeğin içi taranır, arşiv bozukluğu (corruption) olup olmadığı test edilir.
2. **Test Veritabanı Oluşturma:** Geçici bir PostgreSQL veritabanı (`hastane_db_verify_test`) oluşturulur.
3. **Gerçek Restore:** Alınan taze yedek, bu geçici veritabanına `pg_restore` ile geri yüklenir.
4. **Veri Bütünlüğü Kıyaslaması:**
 - `COUNT(*)` işlemi ile ana DB'deki satır sayılarının test DB'siyle birebir uyuşup uyuşmadığı kontrol edilir.
 - `information_schema` taranarak Foreign Key kısıtlamalarının (Constraint) eksiksiz yaratıldığı doğrulanır.
 - İndeksler ve PL/pgSQL fonksiyon sayılarının (Metadata) doğruluğu test edilir.
5. **Temizlik ve Onaylama:** Test DB silinir ve ana veritabanındaki `yedekleme_log` tablosunda o yedeğe ait `dogrulama_durumu = TRUE` olarak işaretlenir.

7. Başarısızlık Tespit ve Uyarı Sistemi

`07_basarisizlik_uyari.sh` scripti, arka planda çalışan bir nöbetçi (Watchdog) görevini görür:

1. **Son Başarılı Yedek Kontrolü:** `yedekleme_log` tablosunu sorgulayarak son başarılı yedeğin yaşını saniye cinsinden hesaplar. Veri 24 saati (86400 sn) geçmişse

"Gecikme (UYARI)", hiç yedek yoksa "Kritik" alarm üretir.

2. **Başarısız İşlem Analizi:** Son 24 saat içinde durumu **basarisiz** olan satırları sorgular. Hatalı işlem sayısını döner.
3. **Anomali Analizi:** Son 24 saatteki Boyut Anomalisi uyarılarını özetler.
4. **Disk Alanı Denetimi:** **df -h** sistem komutuyla fiziksel diskin kullanım oranını hesaplar, %80 aşımında Uyarı, %90 aşımında Kritik Alarm (Sesli macOS bildirimi) tetikler.

8. Denetim Fonksiyonları ve HTML Raporlama

PostgreSQL içinde yöneticilerin SQL komutlarıyla durumu izleyebilmesi için PL/pgSQL kullanılarak özel fonksiyonlar (**fn_yedekleme_istatistik()** , **fn_yedekleme_rapor(N)**) yazılmıştır.

Ayrıca **06_yedekleme_raporlama.sh** scripti bu verileri okuyarak, şirket yöneticilerine sunulabilecek **HTML/CSS tabanlı görsel bir rapor (Dashboard)** üretir.

- Rapor karanlık tema (Dark-theme) üzerine inşa edilmiştir.
- Tepe kısmında metrik kartları (Toplam Yedek Sayısı, DB Boyutu, Başarı Oranı vb.) yer alır.
- Tablo kısmında son 20 yedeklemenin renk kodlu listesi (**badge-ok** , **badge-err**) gösterilir.
- Disk kullanım özeti ile klasör bazlı tüketim (Daily/Weekly/Monthly boyutları) sunulur.

9. Tam Demo Senaryosu

10_tam_otomasyon_demo.sh çalıştırıldığında sistemin bir haftalık operasyonel ömrü birkaç dakika içinde simüle edilir:

Adım	İşlem
1	Hastane Veritabanı oluşturulur, tablolar yapılandırılır ve örnek test verileriyle doldurulur.
2	Otomatik Yedekleme Scripti tetiklenir, yedek alınır, boyut anomalisi olup olmadığı kontrol edilir, başarı durumunda DB'ye log düşülür.
3	Otomatik Yedek Doğrulama tetiklenir; test veritabanı kurulup yedek restore edilir, satır sayılarının uyuştugu gösterilir. DB'de yedeğe "doğrulandı" (verified) işareti atılır.
4	Uyarı ve Denetim Sistemi çalıştırılarak sistem sağlığı denetlenir.
5	Rotasyon devreye girer. Sahte (Mock) eski dosyalar üretilip kural motorunun bu dosyaları diskten otomatik olarak sildiği simüle edilir.
6	Veritabanından derlenen istatistikler modern bir HTML raporuna dönüştürölüp tarayıcıda sunuma hazır hale getirilir.

10. Sonuç ve Değerlendirme

10.1 Elde Edilen Kazanımlar

Bu proje sayesinde aşağıdaki endüstri standardı kazanımlar elde edilmiştir:

- **Veri Güvencesi:** Planlı yedeklemelerin yanında Plain SQL (Clear-text) arşivleriyle farklı senaryolara hazır olunmuştur.
- **Güvenilirlik ve Güven:** "Blind backup" (alınan ancak çalışıp çalışmadığı bilinmeyen yedek) sendromu, tam otomatik restore testleriyle (Verify) aşılmıştır.
- **Sürdürülebilirlik:** Disk dolması riski, akıllı rotasyon (Retention Policy) ile bertaraf edilmiştir.
- **Şeffaflık:** Bash ile PostgreSQL arasındaki güçlü entegrasyon sayesinde tüm operasyonların SQL ile raporlanabilmesi ve HTML Dashboard üretilmesi sağlanmıştır.

10.2 En İyi Uygulamalar (Best Practices)

Sistemi tasarlarken gözetilen kurumsal prensipler:

- **Alerting (Uyarı):** Problemin kullanıcıdan değil, sistemden öğrenilmesi gerektiği (Bildirim entegrasyonu).
- **Format:** Tüm yedeklerin `pg_dump -Fc` kullanılarak hem sıkıştırılmış boyutta olması, hem de sadece istenilen tablonun kurtarılmasına (Selective Restore) izin vermesi.
- **Auditing:** Bir uygulamanın çalışıp çalışmadığının o uygulamaya değil, bağımsız denetleyici fonksiyonlara (`fn_yedekleme_uyari_kontrol`) bırakılması.

Kaynakça

1. PostgreSQL Documentation — Backup and Restore
<https://www.postgresql.org/docs/14/backup.html>
2. PostgreSQL Documentation — pg_dump
<https://www.postgresql.org/docs/14/app-pgdump.html>
3. PostgreSQL Documentation — pg_restore
<https://www.postgresql.org/docs/14/app-pgrestore.html>
4. UNIX/Linux man pages — Bash Scripting and cron
<https://man7.org/linux/man-pages/man1/bash.1.html>
5. EnterpriseDB — Best Practices for PostgreSQL Backup
<https://www.enterprisedb.com/postgresql-backup-and-recovery-best-practices>